In [1]:
```
import pandas as pd

df = pd.read_csv("../data/filteredvehpub.csv")   # adjust path as need
df.head()
df.shape

## read files
```

Out[1]:
```
(256115, 14)
```

In [2]:
```
df.columns

## features and targets
```

Out[2]:
```
Index(['HOUSEID', 'HHFAMINC', 'HHSIZE', 'LIF_CYC', 'CENSUS_R', 'URBA
N',
       'VEHID', 'DRVRCNT', 'MAKE', 'HH_RACE', 'WRKCOUNT', 'HOMEOWN',
       'URBANSIZE', 'VEHYEAR'],
      dtype='object')
```

In [3]:
```
import numpy as np
invalid_values = [-9, -8, -7, -88, 99, "99", "XX", "xx", "XX ", "-88",
df = df.replace(invalid_values, np.nan)
df = df.dropna()

## Remove NaN
```

In [4]:
```
#df["VEHTYPE"] = df["VEHTYPE"].replace([5, 6], np.nan)
#df = df.dropna()
```

In [5]:
```
counts = df["MAKE"].value_counts()
counts.describe()

# Get percentile of the data
```

Out[5]:
```
count       53.000000
mean      4607.245283
std       8163.852477
min         58.000000
25%        286.000000
50%       1580.000000
75%       4639.000000
max      34768.000000
Name: count, dtype: float64
```

In [6]:
```
df["MAKE"] = df["MAKE"].astype(str)   # convert everything to string f
df["MAKE"] = df["MAKE"].str.strip()   # remove whitespace
df["MAKE"] = df["MAKE"].astype(int)   # convert to integer

## Did this because the original data has different variable types, wa
```

```
counts = df["MAKE"].value_counts()
rare_makes = counts[counts < 1743].index
## The number of 1000 can be found in the report


df["MAKE"] = df["MAKE"].where(~df["MAKE"].isin(rare_makes), 98)
df["MAKE"].value_counts()
## Put them all into to other
```

Out[6]:
```
MAKE
12    34768
49    33623
20    31188
37    23829
98    16040
7     12545
35    11684
2      7293
48     6932
23     6896
55     6765
59     4967
18     4852
6      4703
63     4639
30     4608
34     4536
41     4335
42     3642
72     3064
19     2664
22     2659
54     2536
51     1831
14     1798
13     1787
Name: count, dtype: int64
```

In [7]:
```
#df.to_csv("processed_data.csv", index=False)
```

In [8]:
```
df.columns.tolist()
```

Out[8]:
```
['HOUSEID',
 'HHFAMINC',
 'HHSIZE',
 'LIF_CYC',
 'CENSUS_R',
 'URBAN',
 'VEHID',
 'DRVRCNT',
 'MAKE',
 'HH_RACE',
 'WRKCOUNT',
 'HOMEOWN',
 'URBANSIZE',
 'VEHYEAR']
```

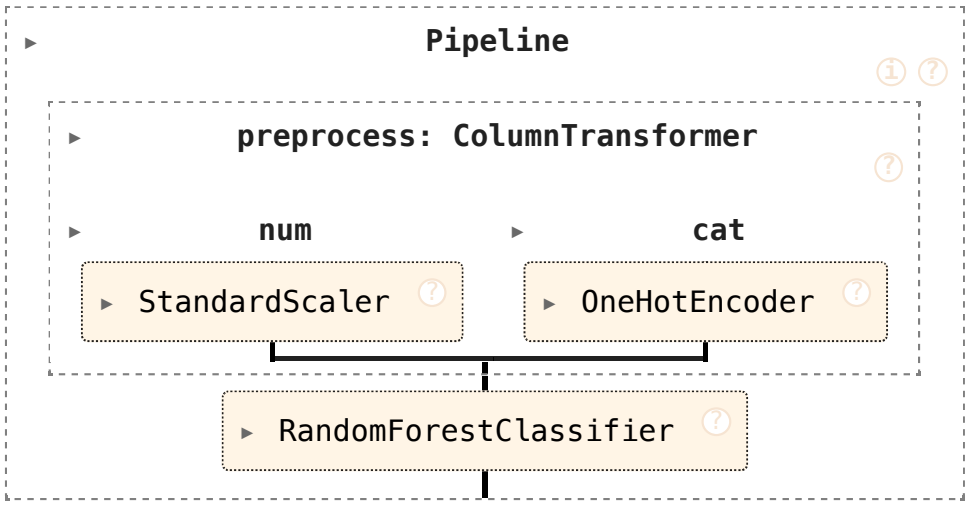
In [9]:
```
feature_cols = [
    "HHSIZE",
    "HHFAMINC",
    "LIF_CYC",
    "CENSUS_R",
    "HH_RACE",
    "HOMEOWN",
    "WRKCOUNT",
    "URBAN",
    "URBANSIZE",
    "DRVRCNT"
]
target_col = "MAKE"
X = df[feature_cols]
y = df[target_col]

# Declare features and targets
```

In [10]:
```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler


numeric_features = ["HHSIZE", "HHFAMINC", "WRKCOUNT", "DRVRCNT"]
categorical_features = ["LIF_CYC", "CENSUS_R", "HH_RACE", "HOMEOWN", "

preprocess = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_fe
    ]
)

## Separate numeric features with categorical features
## Then use standardScaler() for num and One-hot endoing for cat
```

In [11]:
```
from sklearn.pipeline import Pipeline
```

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=300,
    random_state=42,
    n_jobs=-1,           # use all CPU cores
    min_samples_split=4,
    min_samples_leaf=2
)

pipe = Pipeline([
    ("preprocess", preprocess),   # your ColumnTransformer from Step 7
    ("rf", rf)
])

pipe

## Construct the tree and the pipeline
```

Out[11]:




In [12]:

```
## Cross Validation Accuracy

from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score
from tqdm import tqdm
import numpy as np

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores = []

for train_idx, test_idx in tqdm(cv.split(X, y), total=5):
    X_train_cv, X_test_cv = X.iloc[train_idx], X.iloc[test_idx]
    y_train_cv, y_test_cv = y.iloc[train_idx], y.iloc[test_idx]

    # Fit pipeline
    pipe.fit(X_train_cv, y_train_cv)

    # Evaluate accuracy
```

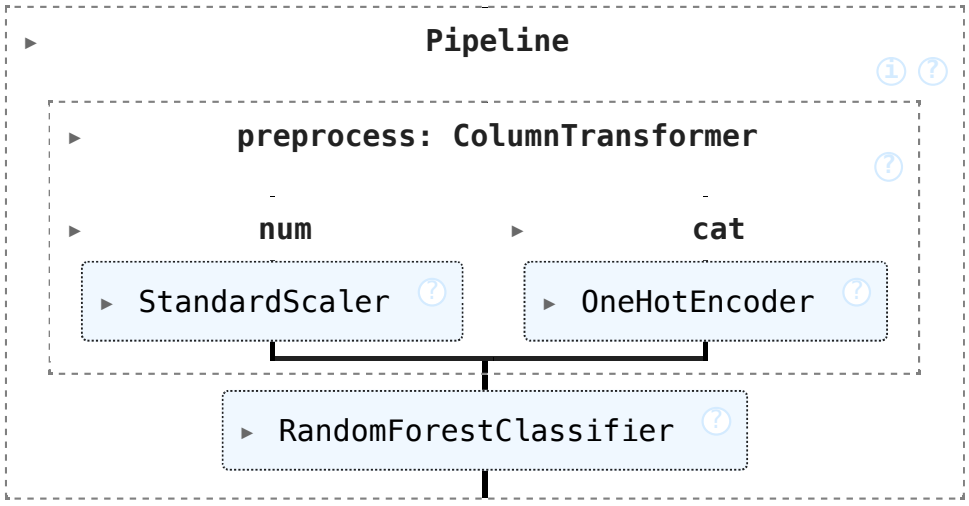
```
    y_pred_cv = pipe.predict(X_test_cv)
    scores.append(accuracy_score(y_test_cv, y_pred_cv))

print("Cross-validated Accuracy:", np.mean(scores), "+/-", np.std(scor
```

```
100%|██████████████████████████████████████████| 5/5 [03:30<00:00, 4
2.00s/it]
Cross-validated Accuracy: 0.17221029061266055 +/- 0.0012839465344754422
```

In [13]:
```
## Fit data into 80% test and 20% train, to train the final model

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0
```

In [14]:
```
pipe.fit(X_train, y_train)
```

Out[14]:

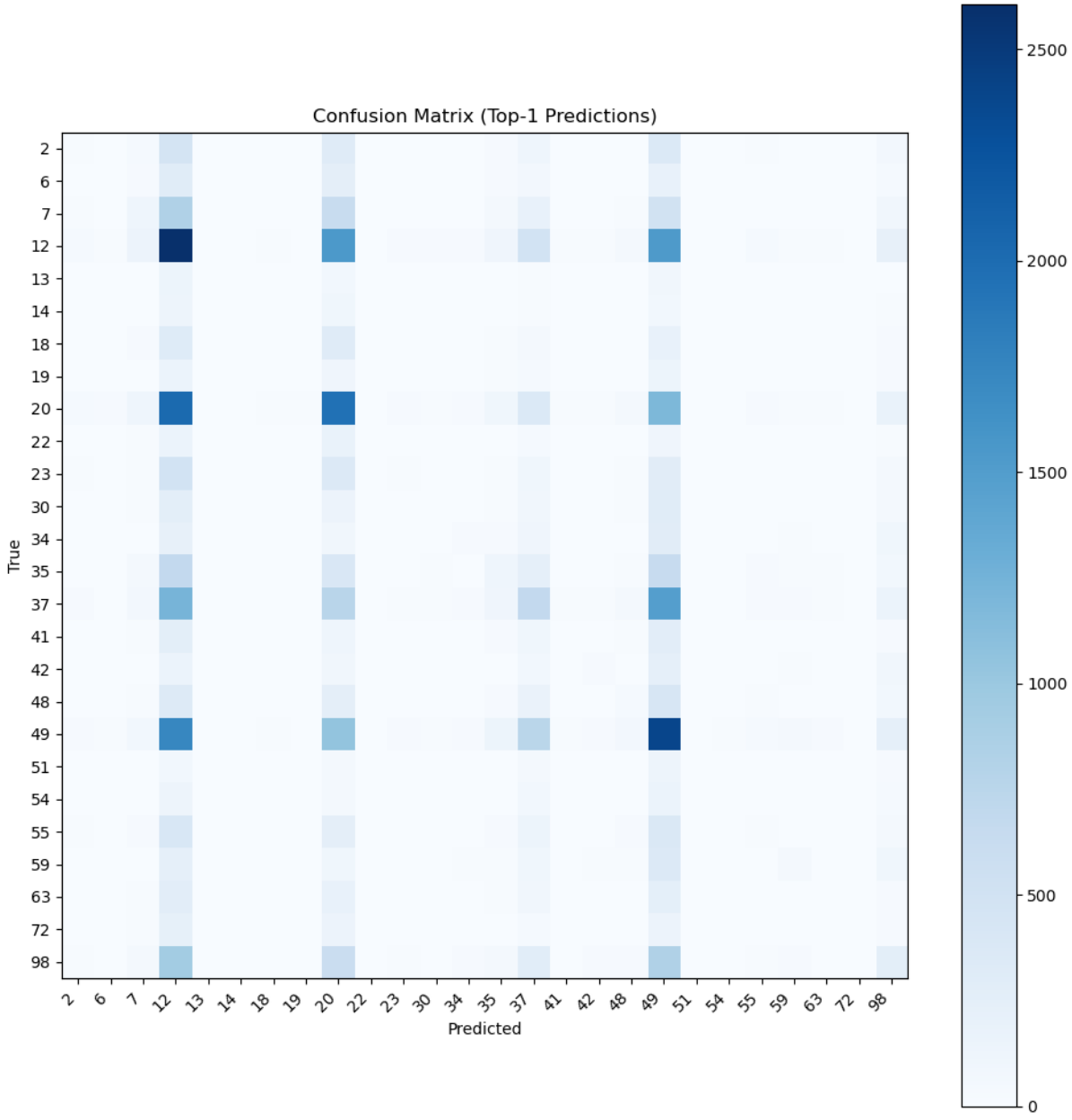



In [35]:
```
## Accuracy of the 80-20 test (seems like people do it for confusion m

from sklearn.metrics import accuracy_score

y_pred = pipe.predict(X_test)

test_acc = accuracy_score(y_test, y_pred)
print("Test Top-1 Accuracy:", test_acc)
```

```
Test Top-1 Accuracy: 0.17050596883510452
```

In [36]:
```
## Confusion Matrix

from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import numpy as np

cm = confusion_matrix(y_test, y_pred)

labels = sorted(y.unique())
```

```
fig, ax = plt.subplots(figsize=(10, 10))
im = ax.imshow(cm, cmap="Blues")
plt.colorbar(im)

ax.set_xticks(np.arange(len(labels)))
ax.set_yticks(np.arange(len(labels)))
ax.set_xticklabels(labels, rotation=45, ha="right")
ax.set_yticklabels(labels)

ax.set_xlabel("Predicted")
ax.set_ylabel("True")
ax.set_title("Confusion Matrix (Top-1 Predictions)")

plt.tight_layout()
plt.show()
```




In [ ]:

In [ ]: